Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	18 July 2026
Team ID	
Project Name	Nutrition Assistant
Maximum Marks	4 Marks

Technical Architecture

The React + Vite client communicates over HTTPS/JSON with an Express.js REST API. Requests pass through Helmet, CORS and rate-limiting middleware, then JWT authentication and Zod validation, before reaching the controller layer. Controllers use Mongoose to read/write MongoDB, and call two external services — the Google Gemini API for the AI Nutrition Chatbot, and Cloudinary for image storage (local disk is used as a fallback when Cloudinary is not configured).

Table-1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	How the user interacts with the application (Web UI)	React.js 19 (Vite), Tailwind CSS v4, Framer Motion, Chart.js
2	Application Logic-1	Core request handling and business logic	Node.js, Express.js (MVC-layered)
3	Application Logic-2	Authentication & role-based authorization logic	JWT (access + refresh tokens), bcryptjs, role-guard middleware
4	Application Logic-3	Tracking, calculation & aggregation logic	Mongoose pre-save hooks; BMI/BMR/TDEE formulas; Zod validators
5	Database	Stores users, foods, recipes, meals, logs, favorites & settings	MongoDB (Mongoose ODM)
6	Cloud Database	Managed cloud database cluster	MongoDB Atlas
7	File Storage	Profile photos, recipe & meal images	Cloudinary (Multer-based local disk fallback)
8	External API-1	Conversational AI nutrition guidance	Google Gemini API (gemini-1.5-flash)
9	External API-2	Not used in the current scope	—
10	Machine Learning Model	No custom-trained model; uses a hosted generative model via API with a rule-based mock fallback	Google Gemini (hosted) + keyword-based fallback responder
11	Infrastructure (Server / Cloud)	Application deployment — client as static site, server as a persistent Node.js API	Cloud hosting (client) + managed Node.js host (server)

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	React.js, Express.js, Node.js, Mongoose, Tailwind CSS	MERN Stack (MongoDB, Express.js, React.js, Node.js)
2	Security Implementations	JWT authentication, bcryptjs hashing, Helmet, rate limiting, role-guard middleware	MERN Stack (MongoDB, Express.js, React.js, Node.js)
3	Scalable Architecture	MVC-layered backend separating routes, middleware, controllers & models	MERN Stack (MongoDB, Express.js, React.js, Node.js)
4	Availability	Static client deployment + persistent API service backed by a managed MongoDB cluster	MERN Stack (MongoDB, Express.js, React.js, Node.js)
5	Performance	Vite build pipeline, Axios API layer, indexed Mongoose text search	MERN Stack (MongoDB, Express.js, React.js, Node.js)

References: https://c4model.com/ | https://www.ibm.com/cloud/architecture | https://aws.amazon.com/architecture